Midterm Mastery

In the first half of this course, I learned how to move from writing simple code on a single computer to building large systems in the cloud. I discovered that a good system is like a building: it needs a strong foundation, a smart layout, and the ability to grow.

1. The Foundation: Atomicity and Threads

Everything starts at the bottom with Threads and Atomic operations. In my experiments with Go, I saw how easily data can be corrupted. When many threads (Goroutines) update a regular counter at the same time, we lose data because the operation is not "atomic."

I learned that we must use hardware-level instructions like `atomic.Uint64` to ensure correctness. This taught me a key lesson: You cannot scale a system if its foundation is broken. If the low-level code is not "thread-safe," the entire system will crash when the load increases.

2. The Middle Layer: API Logic and Data Structures

Once the foundation is safe, we need to handle data efficiently in the Middle Layer. In my assignments, I built REST APIs using the Go/Gin framework.

I learned that choosing the right data structure is a trade-off:

- Standard Maps: Very fast but they crash if two threads write at the same time.
- Mutex (Locks): They protect the data, but they make the system "serial." If one thread holds the lock, others must wait.
- Sync.Map: Better for systems with many "reads" and few "writes."

Through performance testing, I found that the "95th percentile latency" is more important than the average. If our data structures are slow, users will experience "jitter" or delays, even if the server is not busy.

3. The Top Layer: Cloud Scaling and ALB

Finally, I learned about Cloud Architecture on AWS. A single server has limits. When I tested 200 users on one Fargate task, the system reached a "Breaking Point."

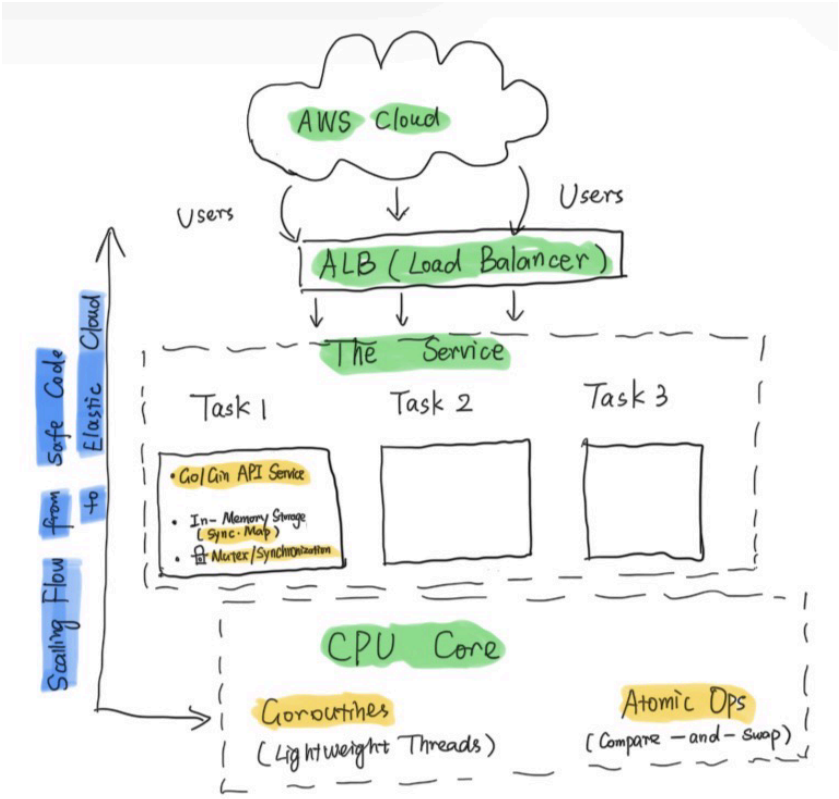
To solve this, I used:

- ALB (Load Balancer): It acts as a traffic conductor. It sends requests to different servers so no single one is overwhelmed.
- Auto Scaling: It automatically adds more servers when the CPU usage is high.

I observed that "Horizontal Scaling" (adding more servers) is much better than "Vertical Scaling" (making one server bigger). It provides high availability. If one server fails, the ALB simply sends traffic to others.

My Thoughts

The logic between these topics is a chain:

1. Correctness (Bottom): Atomic operations ensure the data is right.
2. Efficiency (Middle): Good data structures ensure the API responds fast.
3. Scalability (Top): ALB and Auto Scaling ensure the system survives millions of users.

If any part of this chain fails, the whole system fails. This "Full-Stack" view is the most valuable thing I have learned so far.